TALLER - TABLERO DE MEDICAMENTOS DISPENSADOS 2020-2021

Python + Dash + SQL (SQLite)

Análisis para una EPS

Base de datos: medicamentos.db | Tabla: dispensacion

Enlace público del tablero: PENDIENTE DE PUBLICAR EN RENDER

Nota: el paquete entregado queda listo para desplegar; la URL pública se obtiene al conectar el repositorio con Render.

1. Objetivo del tablero

El objetivo es construir un tablero interactivo que permita estudiar las dispensaciones de medicamentos entre 2020 y 2021, medir la población atendida, el número de fórmulas, los costos y los patrones de dispensación. El tablero se conecta directamente a una base SQLite y ejecuta consultas SQL desde Python, mientras Dash y Plotly presentan los indicadores y gráficos de forma interactiva.

2. Fuente y estructura de los datos

La base utilizada es medicamentos.db. Contiene una tabla llamada dispensacion con 28.482 registros. Las variables principales usadas en el tablero son: id, formula, fecha_entrega, regional_caf, departamento_caf, municipio_caf, descripcion, grupo_fco_economico, cantidad, precio, costo_total, pbs y tipo_entrega.

Variable	Uso	Cálculo / visualización
id	Persona	COUNT(DISTINCT id)
formula	Fórmula dispensada	COUNT(DISTINCT formula)
costo_total	Costo monetario	SUM(costo_total)
fecha_entrega	Evolución temporal	Agrupación por año-mes
descripcion	Medicamento	Top por costo total
pbs	Plan de beneficios	Costo PBS vs No PBS
municipio_caf	Lugar de dispensación	Costo por municipio
tipo_entrega	Modalidad	Costo por tipo de entrega
grupo_fco_economico	Filtro farmacológico	Filtro y análisis ANTIDIABÉTICOS
regional_caf	Regional	Filtro interactivo

3. Indicadores principales

Sin aplicar filtros, la base registra 1.000 personas con dispensaciones y 10.005 fórmulas distintas. El costo total observado es aproximadamente $745,2 millones. El costo promedio de una fórmula se calcula agregando primero todos los renglones pertenecientes a una misma fórmula y después promediando el costo total de esas fórmulas; el resultado global es aproximadamente $74.478 por fórmula.

Personas	Fórmulas distintas	Costo promedio por fórmula
1.000	**10.005**	**$74.478**

4. Lógica de construcción del tablero

La aplicación usa una arquitectura sencilla: (1) Dash recibe los valores seleccionados en los filtros; (2) se construye dinámicamente la cláusula WHERE; (3) las consultas SQL se ejecutan sobre SQLite; (4) los resultados se cargan en DataFrames de pandas; y (5) Plotly genera las visualizaciones. De esta forma, el tablero no necesita cargar toda la base en memoria en cada interacción y mantiene explícito el uso de SQL solicitado en el taller.

4.1 Filtros

Se implementaron cuatro filtros globales: grupo farmacológico, año, mes de dispensación y regional CAF. Al cambiar un filtro se recalculan los tres KPI y las visualizaciones principales.

4.2 Visualizaciones incluidas

- Número de personas con dispensaciones: COUNT(DISTINCT id).
- Número de fórmulas distintas: COUNT(DISTINCT formula).
- Costo promedio de fórmula: promedio de SUM(costo_total) agrupado por formula.
- Dispensaciones en el tiempo: conteo mensual de registros.
- Top 10 de medicamentos por mayor costo total: SUM(costo_total) por descripcion.
- Costo según PBS / No PBS: agregación de costo_total por pbs.
- Costo por municipio de dispensación: agregación por municipio_caf.
- Costo según tipo de entrega: agregación por tipo_entrega.
- Filtros por grupo farmacológico, año, mes y regional CAF.

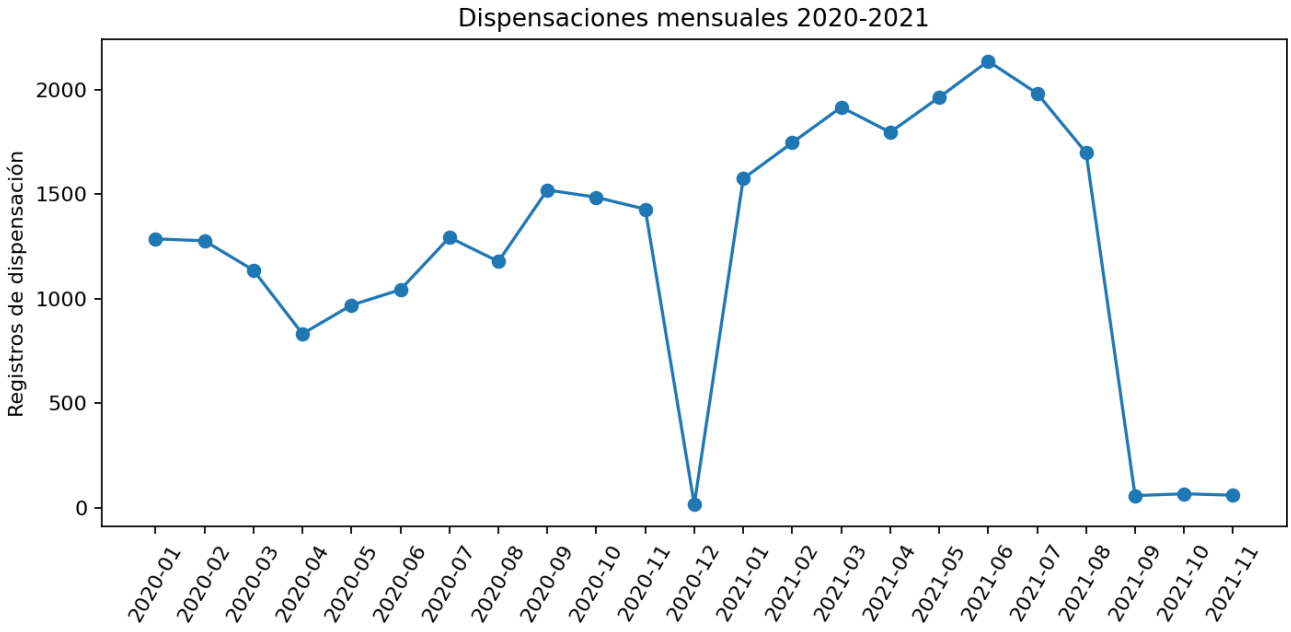


Figura 1. Evolución de los registros de dispensación por mes.

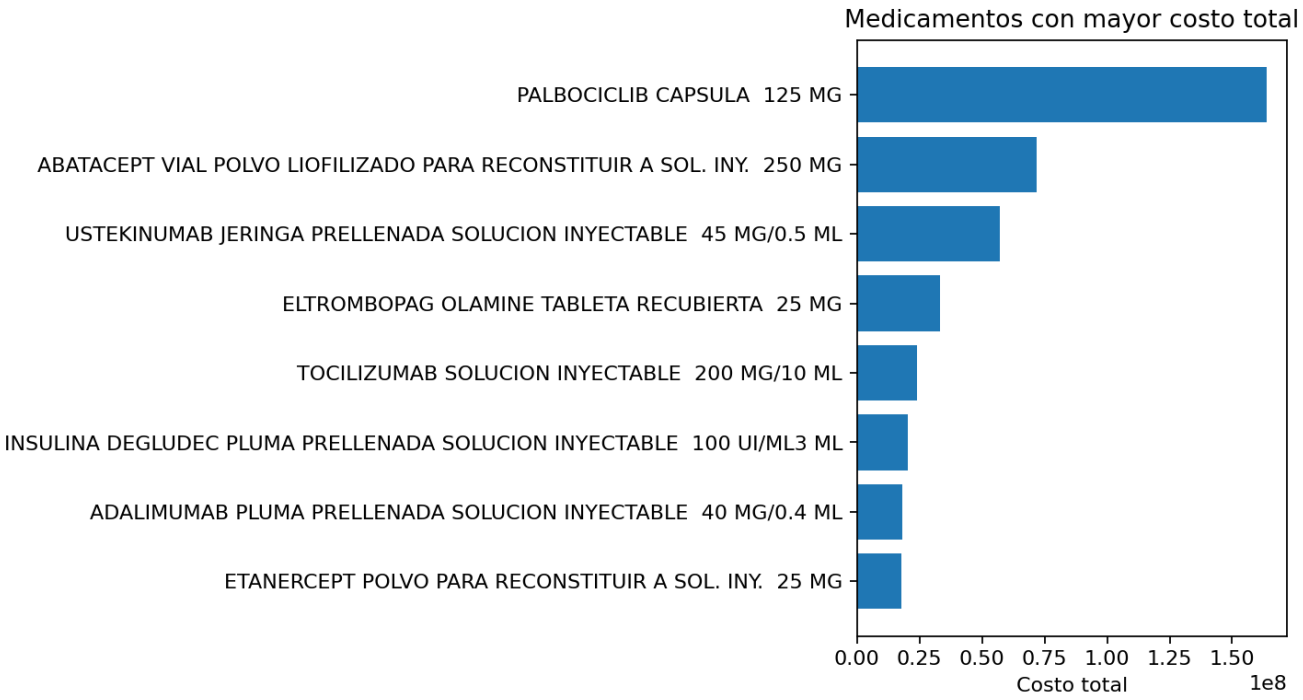


Figura 2. Medicamentos con mayor costo total en la base.

5. Hallazgos generales

El gasto está altamente concentrado en algunos medicamentos. El medicamento con mayor costo acumulado en toda la base es PALBOCICLIB CÁPSULA 125 MG, con aproximadamente $163,7 millones. En segundo lugar aparece ABATACEPT 250 MG, con cerca de $71,9 millones, seguido por USTEKINUMAB 45 MG/0,5 ML, con aproximadamente $57,0 millones.

Por cobertura, los medicamentos No PBS acumulan aproximadamente $452,8 millones, mientras los PBS acumulan alrededor de $292,3 millones. Por tipo de entrega, la modalidad domiciliaria concentra cerca de $551,9 millones y la presencial alrededor de $191,4 millones. Estos resultados son descriptivos y deben interpretarse en función del número de fórmulas, cantidades y mezcla de medicamentos de cada grupo.

6. Análisis del grupo ANTIDIABÉTICOS

Existe una limitación importante de calidad de datos: en 2020 el campo grupo_fco_economico está vacío en todos los registros. En 2021 sí existe clasificación farmacológica y se identifican 648 registros de ANTIDIABÉTICOS, correspondientes a 482 fórmulas distintas. Por tanto, no es metodológicamente correcto comparar 2020 y 2021 usando directamente el campo grupo_fco_economico.

Para responder de forma aproximada a la pregunta sobre el crecimiento del costo, se identificaron las descripciones clasificadas como ANTIDIABÉTICOS en 2021 y se buscaron esas mismas descripciones en 2020. Con esta aproximación, el costo pasa de aproximadamente $33,98 millones en 2020 a $38,92 millones en 2021, equivalente a un aumento estimado de 14,6%. Esta cifra debe presentarse como una estimación, no como una comparación directa de categorías, debido al faltante de clasificación en 2020.

Pregunta	Resultado
¿El costo aumentó?	Estimación: +14,6% entre 2020 y 2021, usando equivalencia de descripciones.
Medicamento más costoso del grupo	INSULINA DEGLUDEC PLUMA PRELLENADA 100 UI/ML 3 ML, con $8.019.243 en 2021.
Costo promedio de una fórmula	Aproximadamente $80.749 por fórmula antidiabética en 2021.

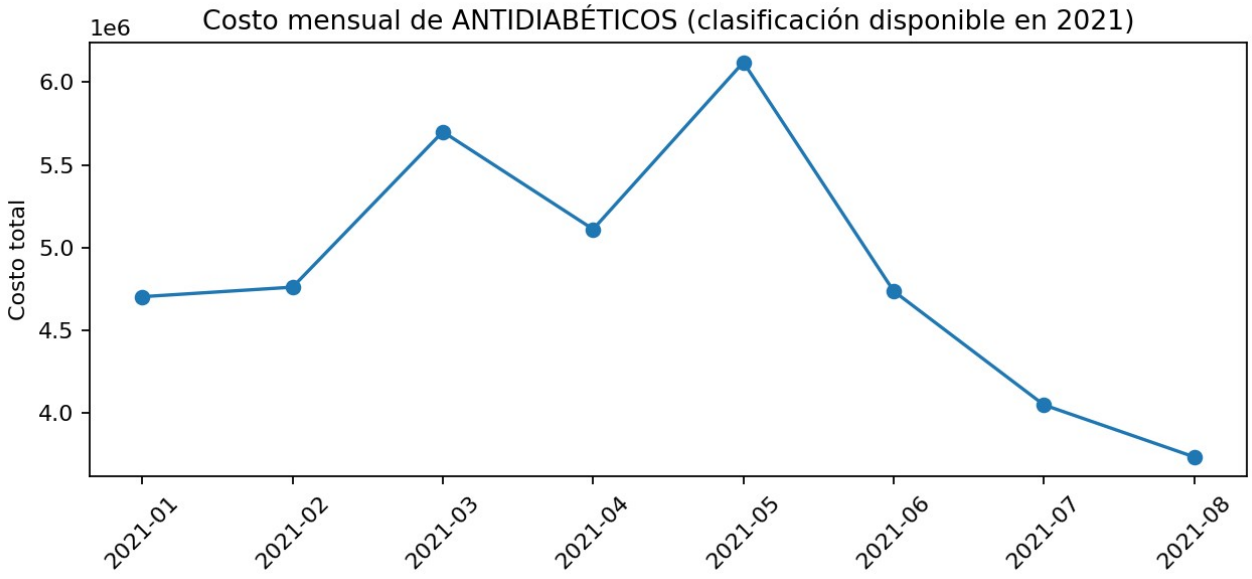


Figura 3. Costo mensual de antidiabéticos según la clasificación disponible en 2021.

7. Consultas SQL fundamentales

Personas y fórmulas

```
SELECT COUNT(DISTINCT id), COUNT(DISTINCT formula)
FROM dispensacion;
```

Costo promedio por fórmula

```
SELECT AVG(costo_formula)
FROM (
  SELECT formula, SUM(costo_total) AS costo_formula
  FROM dispensacion
  GROUP BY formula
);
```

Top de medicamentos

```
SELECT descripcion, SUM(costo_total) AS costo_total
FROM dispensacion
GROUP BY descripcion
ORDER BY costo_total DESC
LIMIT 10;
```

ANTIDIABÉTICOS 2021

```
SELECT descripcion, SUM(costo_total) AS costo_total
FROM dispensacion
WHERE grupo_fco_economico='ANTIDIABETICOS'
GROUP BY descripcion
ORDER BY costo_total DESC;
```

8. Despliegue

El proyecto incluye requirements.txt, Procfile y render.yaml. Para obtener la URL pública, la carpeta debe subirse a GitHub y conectarse a Render como un Web Service. El comando de construcción es pip install -r requirements.txt y el comando de inicio es gunicorn app:server. Render genera una URL pública que debe copiarse en la portada de este informe antes de la entrega final.

9. Conclusiones

El tablero permite responder los once requerimientos funcionales de forma interactiva usando SQL como capa de consulta y Dash como capa de visualización. El principal hallazgo de calidad es la ausencia de grupo_fco_economico en 2020, lo cual afecta cualquier comparación directa por grupo farmacológico. Para ANTIDIABÉTICOS se documenta una aproximación reproducible basada en las descripciones clasificadas en 2021, con un incremento estimado del costo de 14,6%, un costo promedio por fórmula de aproximadamente $80.749 y la insulina degludec como el medicamento de mayor costo acumulado del grupo en 2021.

ANEXO A. Código completo de la aplicación

Archivo: app.py. El siguiente código constituye el tablero Dash completo y ejecuta las consultas SQL sobre medicamentos.db.

```
import os
import sqlite3
import pandas as pd
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import plotly.graph_objects as go


DB_PATH = os.getenv("DB_PATH", os.path.join(os.path.dirname(__file__), "medicamentos.db"))



def sql_df(query, params=None):
    with sqlite3.connect(DB_PATH) as con:
        return pd.read_sql_query(query, con, params=params or [])



def money(x):
    if x is None or pd.isna(x):
        return "$0"
    return f"${x:,.0f}".replace(",", ".")



def get_filter_options():
    groups = sql_df("""
        SELECT DISTINCT grupo_fco_economico AS value
        FROM dispensacion
        WHERE TRIM(COALESCE(grupo_fco_economico,'')) <> ''
        ORDER BY value
    """)["value"].tolist()
    regions = sql_df("""
        SELECT DISTINCT regional_caf AS value
        FROM dispensacion
        WHERE TRIM(COALESCE(regional_caf,'')) <> ''
        ORDER BY value
    """)["value"].tolist()
    years = sql_df("""
        SELECT DISTINCT CAST(substr(fecha_entrega,1,4) AS INTEGER) AS value
        FROM dispensacion ORDER BY value
    """)["value"].tolist()
    return groups, regions, years



def where_clause(group, year, month, region):
    clauses = ["1=1"]
    params = []
    if group:
        clauses.append("grupo_fco_economico = ?")
        params.append(group)
    if year:
        clauses.append("substr(fecha_entrega,1,4) = ?")
        params.append(str(year))
    if month:
        clauses.append("CAST(substr(fecha_entrega,6,2) AS INTEGER) = ?")
        params.append(int(month))
    if region:
        clauses.append("regional_caf = ?")
        params.append(region)
    return " AND ".join(clauses), params



groups, regions, years = get_filter_options()
months = [
    {"label": n, "value": i}
    for i, n in enumerate(
        ["Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio", "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"],
        start=1,
    )
]

app = Dash(__name__)
server = app.server
app.title = "Medicamentos dispensados EPS"


CARD = {
    "background": "white",
    "borderRadius": "14px",
    "padding": "18px",
    "boxShadow": "0 2px 10px rgba(0,0,0,.08)",
```

```
}

app.layout = html.Div(
    style={"fontFamily": "Arial, sans-serif", "background": "#f4f6f9", "minHeight": "100vh", "padding": "22px"},
    children=[
        html.H1("Tablero de medicamentos dispensados 2020-2021", style={"marginBottom": "4px"}),
        html.P("EPS - análisis de dispensación, costos y segmentación", style={"color": "#5f6b7a", "marginTop": 0}),
        html.Div(
            style={**CARD, "marginBottom": "18px"},
            children=[
                html.H3("Filtros"),
                html.Div(
                    style={"display": "grid", "gridTemplateColumns": "repeat(4, minmax(180px,1fr))", "gap": "12px"},
                    children=[
                        dcc.Dropdown(groups, placeholder="Grupo farmacológico", id="f-group", clearable=True),
                        dcc.Dropdown(years, placeholder="Año", id="f-year", clearable=True),
                        dcc.Dropdown(months, placeholder="Mes", id="f-month", clearable=True),
                        dcc.Dropdown(regions, placeholder="Regional CAF", id="f-region", clearable=True),
                    ],
                ),
            ],
        ),
        html.Div(
            style={"display": "grid", "gridTemplateColumns": "repeat(3,1fr)", "gap": "14px", "marginBottom": "18px"},
            children=[
                html.Div([html.Div("Personas con dispensaciones", style={"color": "#6b7280"}), html.H2(id="kpi-personas")], style=CARD),
                html.Div([html.Div("Fórmulas distintas", style={"color": "#6b7280"}), html.H2(id="kpi-formulas")], style=CARD),
                html.Div([html.Div("Costo promedio por fórmula", style={"color": "#6b7280"}), html.H2(id="kpi-promedio")], style=CARD),
            ],
        ),
        html.Div(
            style={"display": "grid", "gridTemplateColumns": "1.3fr 1fr", "gap": "14px", "marginBottom": "14px"},
            children=[
                html.Div(dcc.Graph(id="g-tiempo"), style=CARD),
                html.Div(dcc.Graph(id="g-top"), style=CARD),
            ],
        ),
        html.Div(
            style={"display": "grid", "gridTemplateColumns": "repeat(3,1fr)", "gap": "14px", "marginBottom": "18px"},
            children=[
                html.Div(dcc.Graph(id="g-pbs"), style=CARD),
                html.Div(dcc.Graph(id="g-municipio"), style=CARD),
                html.Div(dcc.Graph(id="g-entrega"), style=CARD),
            ],
        ),
        html.Div(
            style={**CARD, "marginTop": "18px"},
            children=[
                html.H2("Análisis especial: ANTIDIABÉTICOS"),
                html.P(
                    "Nota metodológica: en 2020 el campo grupo_fco_economico está vacío. Para comparar 2020 con 2021 se usa como aproximación el conjunto de descripciones que sí están
clasificadas como ANTIDIABETICOS en 2021 y se buscan esas mismas descripciones en 2020.",
                    style={"color": "#5f6b7a"},
                ),
                html.Div(
                    style={"display": "grid", "gridTemplateColumns": "repeat(3,1fr)", "gap": "14px", "marginBottom": "12px"},
                    children=[
                        html.Div([html.Div("Variación estimada 2020-2021"), html.H2(id="anti-var")], style={**CARD, "boxShadow": "none", "border": "1px solid #e5e7eb"}),
                        html.Div([html.Div("Medicamento de mayor costo (2021)"), html.H4(id="anti-top")], style={**CARD, "boxShadow": "none", "border": "1px solid #e5e7eb"}),
                        html.Div([html.Div("Costo promedio por fórmula (2021)"), html.H2(id="anti-prom")], style={**CARD, "boxShadow": "none", "border": "1px solid #e5e7eb"}),
                    ],
                ),
                dcc.Graph(id="anti-trend"),
            ],
        ),
        html.P("Fuente: medicamentos.db, tabla dispensacion.", style={"color": "#7b8491", "fontSize": "12px", "marginTop": "18px"}),
    ],
)



@app.callback(
    Output("kpi-personas", "children"),
    Output("kpi-formulas", "children"),
    Output("kpi-promedio", "children"),
    Output("g-tiempo", "figure"),
    Output("g-top", "figure"),
    Output("g-pbs", "figure"),
    Output("g-municipio", "figure"),
    Output("g-entrega", "figure"),
    Output("anti-var", "children"),
    Output("anti-top", "children"),
    Output("anti-prom", "children"),
    Output("anti-trend", "figure"),
    Input("f-group", "value"),
    Input("f-year", "value"),
    Input("f-month", "value"),
    Input("f-region", "value"),
)
def update_dashboard(group, year, month, region):
    where, params = where_clause(group, year, month, region)

    kpi = sql_df(f"""
        SELECT COUNT(DISTINCT id) personas, COUNT(DISTINCT formula) formulas
        FROM dispensacion WHERE {where}
```

```
""", params).iloc[0]
prom = sql_df(f"""
    SELECT AVG(costo_formula) promedio
    FROM (
        SELECT formula, SUM(costo_total) costo_formula
        FROM dispensacion
        WHERE {where} AND formula IS NOT NULL
        GROUP BY formula
    )
""", params).iloc[0]["promedio"]


tiempo = sql_df(f"""
    SELECT substr(fecha_entrega,1,7) periodo,
           COUNT(*) dispensaciones,
           SUM(costo_total) costo_total
    FROM dispensacion
    WHERE {where}
    GROUP BY periodo ORDER BY periodo
""", params)
fig_t = px.line(tiempo, x="periodo", y="dispensaciones", markers=True, title="Dispensaciones en el tiempo")
fig_t.update_layout(xaxis_title="Mes", yaxis_title="Número de dispensaciones", margin=dict(l=20,r=20,t=55,b=20))


top = sql_df(f"""
    SELECT descripcion, SUM(costo_total) costo_total
    FROM dispensacion
    WHERE {where}
    GROUP BY descripcion
    ORDER BY costo_total DESC LIMIT 10
""", params).sort_values("costo_total")
fig_top = px.bar(top, x="costo_total", y="descripcion", orientation="h", title="Top 10 medicamentos por costo total")
fig_top.update_layout(xaxis_title="Costo total", yaxis_title="", margin=dict(l=20,r=20,t=55,b=20))


pbs = sql_df(f"""
    SELECT CASE WHEN TRIM(COALESCE(pbs,''))='' THEN 'SIN DATO' ELSE pbs END pbs,
           SUM(costo_total) costo_total
    FROM dispensacion WHERE {where}
    GROUP BY 1 ORDER BY costo_total DESC
""", params)
fig_pbs = px.pie(pbs, names="pbs", values="costo_total", title="Costo según PBS / No PBS", hole=.45)


muni = sql_df(f"""
    SELECT CASE WHEN TRIM(COALESCE(municipio_caf,''))='' THEN 'SIN DATO' ELSE municipio_caf END municipio_caf,
           SUM(costo_total) costo_total
    FROM dispensacion WHERE {where}
    GROUP BY 1 ORDER BY costo_total DESC LIMIT 12
""", params).sort_values("costo_total")
fig_m = px.bar(muni, x="costo_total", y="municipio_caf", orientation="h", title="Costo por municipio de dispensación")
fig_m.update_layout(xaxis_title="Costo total", yaxis_title="")


ent = sql_df(f"""
    SELECT CASE WHEN TRIM(COALESCE(tipo_entrega,''))='' THEN 'SIN DATO' ELSE tipo_entrega END tipo_entrega,
           SUM(costo_total) costo_total
    FROM dispensacion WHERE {where}
    GROUP BY 1 ORDER BY costo_total DESC
""", params)
fig_e = px.bar(ent, x="tipo_entrega", y="costo_total", title="Costo según tipo de entrega")
fig_e.update_layout(xaxis_title="Tipo de entrega", yaxis_title="Costo total")


# Análisis ANTIDIABÉTICOS. Se deja fijo e independiente de los filtros globales.
anti_year = sql_df("""
    WITH anti AS (
        SELECT DISTINCT descripcion FROM dispensacion
        WHERE grupo_fco_economico='ANTIDIABETICOS'
    )
    SELECT substr(fecha_entrega,1,4) anio, SUM(costo_total) costo_total
    FROM dispensacion
    WHERE descripcion IN (SELECT descripcion FROM anti)
    GROUP BY anio ORDER BY anio
""")
c20 = anti_year.loc[anti_year["anio"]=="2020", "costo_total"]
c21 = anti_year.loc[anti_year["anio"]=="2021", "costo_total"]
variacion = ((c21.iloc[0] / c20.iloc[0]) - 1) * 100 if len(c20) and len(c21) and c20.iloc[0] else None


anti_top_df = sql_df("""
    SELECT descripcion, SUM(costo_total) costo_total
    FROM dispensacion
    WHERE grupo_fco_economico='ANTIDIABETICOS'
    GROUP BY descripcion ORDER BY costo_total DESC LIMIT 1
""")
anti_top_text = "Sin datos"
if not anti_top_df.empty:
    anti_top_text = f"{anti_top_df.iloc[0]['descripcion']} - {money(anti_top_df.iloc[0]['costo_total'])}"


anti_prom_df = sql_df("""
    SELECT AVG(costo_formula) promedio
    FROM (
        SELECT formula, SUM(costo_total) costo_formula
        FROM dispensacion
        WHERE grupo_fco_economico='ANTIDIABETICOS' AND formula IS NOT NULL
        GROUP BY formula
    )
```

```
    """)
    anti_prom = anti_prom_df.iloc[0]["promedio"]


    anti_month = sql_df("""
        SELECT substr(fecha_entrega,1,7) periodo, SUM(costo_total) costo_total
        FROM dispensacion
        WHERE grupo_fco_economico='ANTIDIABETICOS'
        GROUP BY periodo ORDER BY periodo
    """)
    anti_fig = px.line(anti_month, x="periodo", y="costo_total", markers=True, title="Costo mensual de ANTIDIABÉTICOS (clasificación disponible en 2021)")
    anti_fig.update_layout(xaxis_title="Mes", yaxis_title="Costo total")


    return (
        f"{int(kpi['personas']):,}".replace(",", "."),
        f"{int(kpi['formulas']):,}".replace(",", "."),
        money(prom),
        fig_t,
        fig_top,
        fig_pbs,
        fig_m,
        fig_e,
        f"{variacion:+.1f}% (estimada)" if variacion is not None else "No calculable",
        anti_top_text,
        money(anti_prom),
        anti_fig,
    )



if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=int(os.getenv("PORT", 8050)))
```